

Here are **detailed, structured notes on NumPy** (clear and interview-ready):

NumPy – Detailed Notes

1. Introduction

NumPy (Numerical Python) is a powerful Python library used for numerical computations. It provides support for large, multi-dimensional arrays and matrices along with mathematical functions to operate on them.

It is the foundation for libraries like Pandas, Scikit-learn, and TensorFlow.

2. Why NumPy is Needed

Normal Python lists:

- Slower for numerical operations
- Consume more memory
- Not optimized for matrix calculations

NumPy:

- Faster (implemented in C)
 - Uses less memory
 - Supports vectorized operations
 - Supports multi-dimensional arrays
-

3. Importing NumPy

```
import numpy as np
```

`np` is an alias (standard convention).

4. NumPy Arrays

Creating Arrays

```
import numpy as np
```

```
a = np.array([1, 2, 3])    # 1D array  
b = np.array([[1, 2], [3, 4]]) # 2D array
```

Other Array Creation Methods

```
np.zeros((2, 2))    # 2x2 array of zeros  
np.ones((3, 3))    # 3x3 array of ones  
np.arange(0, 10, 2) # numbers from 0 to 10 with step 2  
np.linspace(0, 1, 5) # 5 evenly spaced numbers between 0 and 1  
np.eye(3)          # Identity matrix  
np.random.rand(3, 3) # Random values
```

5. Array Attributes

```
a.ndim    # Number of dimensions  
a.shape   # Shape of array (rows, columns)  
a.size    # Total number of elements  
a.dtype   # Data type  
a.itemsize # Size of one element (in bytes)
```

Example:

```
a = np.array([[1,2,3],[4,5,6]])  
print(a.shape) # (2,3)
```

6. Indexing and Slicing

Indexing

```
a[0]  
a[1][2]
```

Slicing

```
a[0:2]
a[:, 1]    # All rows, second column
a[1, :]    # Second row
```

7. Array Operations

NumPy supports **vectorized operations** (no need for loops).

```
a + 2
a - 1
a * 3
a / 2
```

Element-wise operations:

```
a1 + a2
a1 * a2
```

8. Mathematical Functions

```
np.sum(a)
np.mean(a)
np.max(a)
np.min(a)
np.std(a)
np.sqrt(a)
np.power(a, 2)
```

9. Reshaping Arrays

```
a.reshape(2, 3)
a.flatten()
a.ravel()
```

Reshaping changes structure without changing data.

10. Broadcasting

Broadcasting allows operations between arrays of different shapes.

Example:

```
a = np.array([1,2,3])  
b = 5  
print(a + b)
```

Output:

```
[6 7 8]
```

NumPy automatically adjusts dimensions.

11. Copy vs View

```
b = a.view() # Shallow copy  
c = a.copy() # Deep copy
```

- View → Changes affect original
 - Copy → Independent
-

12. Aggregation Functions

```
a.sum(axis=0) # Column-wise sum  
a.sum(axis=1) # Row-wise sum
```

13. Advantages of NumPy

- High performance
- Efficient memory usage
- Easy matrix operations

- Backbone of Data Science
-

14. Real-Time Use Cases

- Data Science
 - Machine Learning
 - Image Processing
 - Scientific Computing
 - Financial Modeling
-

Interview Important Points

- NumPy arrays are homogeneous (same data type).
 - Faster than Python lists because implemented in C.
 - Supports vectorization.
 - Uses contiguous memory allocation.
-

If you cannot write a small matrix addition or slicing program confidently, revise arrays again before moving to Pandas or ML.